VIT
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

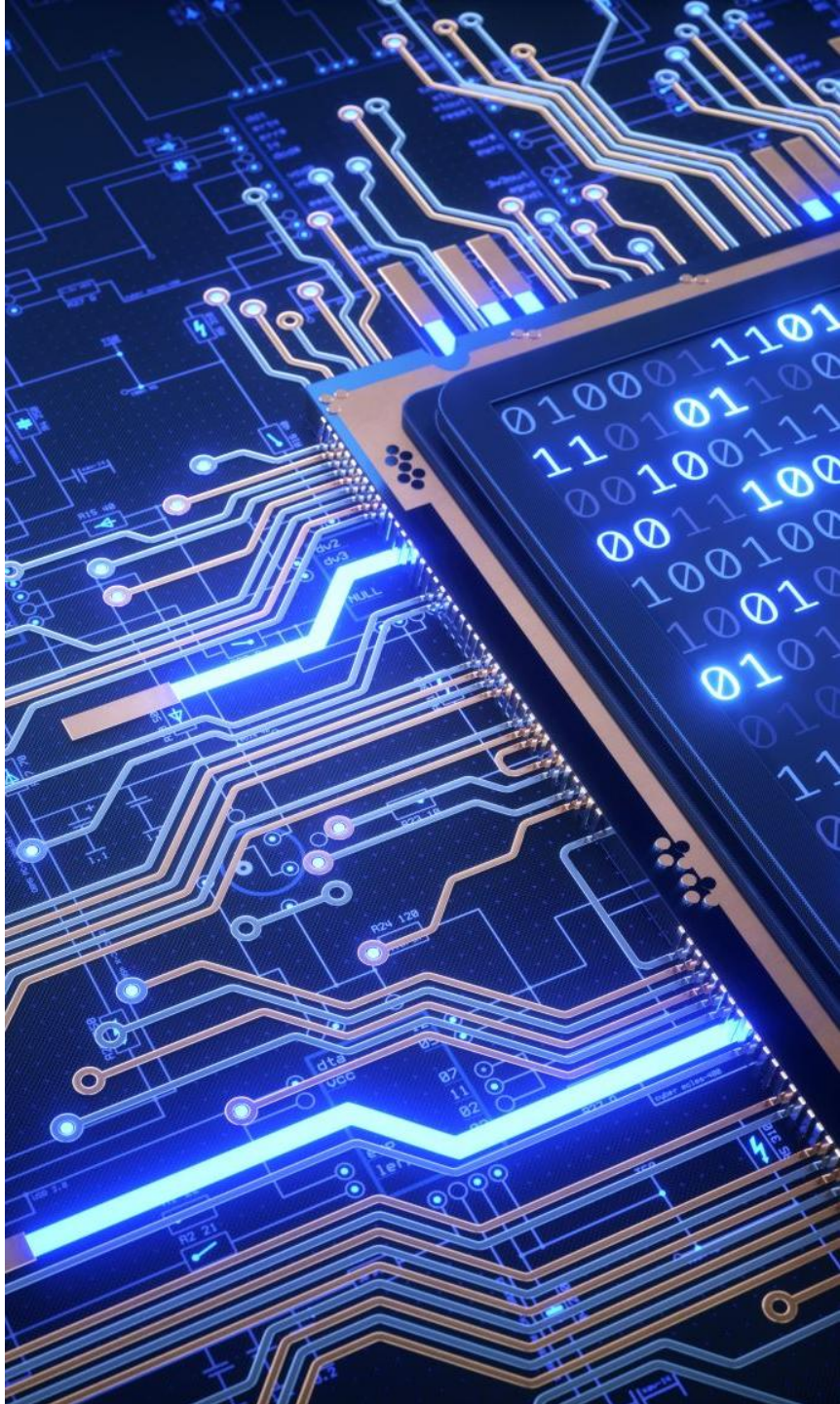
# Microprocessors and Microcontrollers Lab
# BECE204P


## Dr Vishal Gupta

Assistant Professor

School of Electronics Engineering

VIT Vellore, Tamil Nadu, India

# Assembly Language Programming

```
ORG 0000H          ; Origin directive, setting the starting address of the program to 0000h
MOV A, #50H        ; Move immediate value 50h into register A
MOV B, #40H        ; Move immediate value 40h into register B
ADD A, B           ; Add the contents of registers A and B, result stored in A
HALT: SJMP HALT    ; Label for the HALT instruction
                   ; Unconditional jump (infinite loop), causing the program to halt
END                ; End directive, indicating the end of the program
```

HALT instruction halts the central processing unit (CPU) until the next external interrupt is fired.

# Assembly Language Programming

```
ORG 0000h       ; Origin directive, setting the starting address of the program to 0000h
MOV A, #00h     ; Move the immediate value 00h (zero) into register A.
x: MOV P1, A    ; Label 'x' for reference
                ; Move the contents of register A to the P1 port
CPL A           ; Complement (invert) the bits in register A
JMP x           ; Unconditional jump to the label 'x', creating an infinite loop
END             ; End directive, indicating the end of the program
```

# Assembly Language Programming

- **Assembly language instruction includes**
  - A mnemonic
  - one or two operands

**An Assembly language instruction consists of four fields:**

[label:] Mnemonic [operands] [;comment]

# Assembly Language Programming

```
ORG   0H              ;start(origin) at location
0
MOV   R5, #25H        ;load 25H into R5
MOV   R7, #34H        ;load 34H in
MOV   A, #0           ;load 0 into
ADD   A, R5           ;add content
                      ;now A = A +
ADD   A, R7           ;add contents of R7 to A
                      ;now A = A + R7
ADD   A, #12H         ;add to A value 12H
                      ;now A = A + 12H
HERE: SJMP HERE       ;stay in this loop
END                   ;en
```

Directives do not generate any machine code and are used only by the assembler

Mnemonics produce opcodes

The label field allows the program to refer to a line of code by name

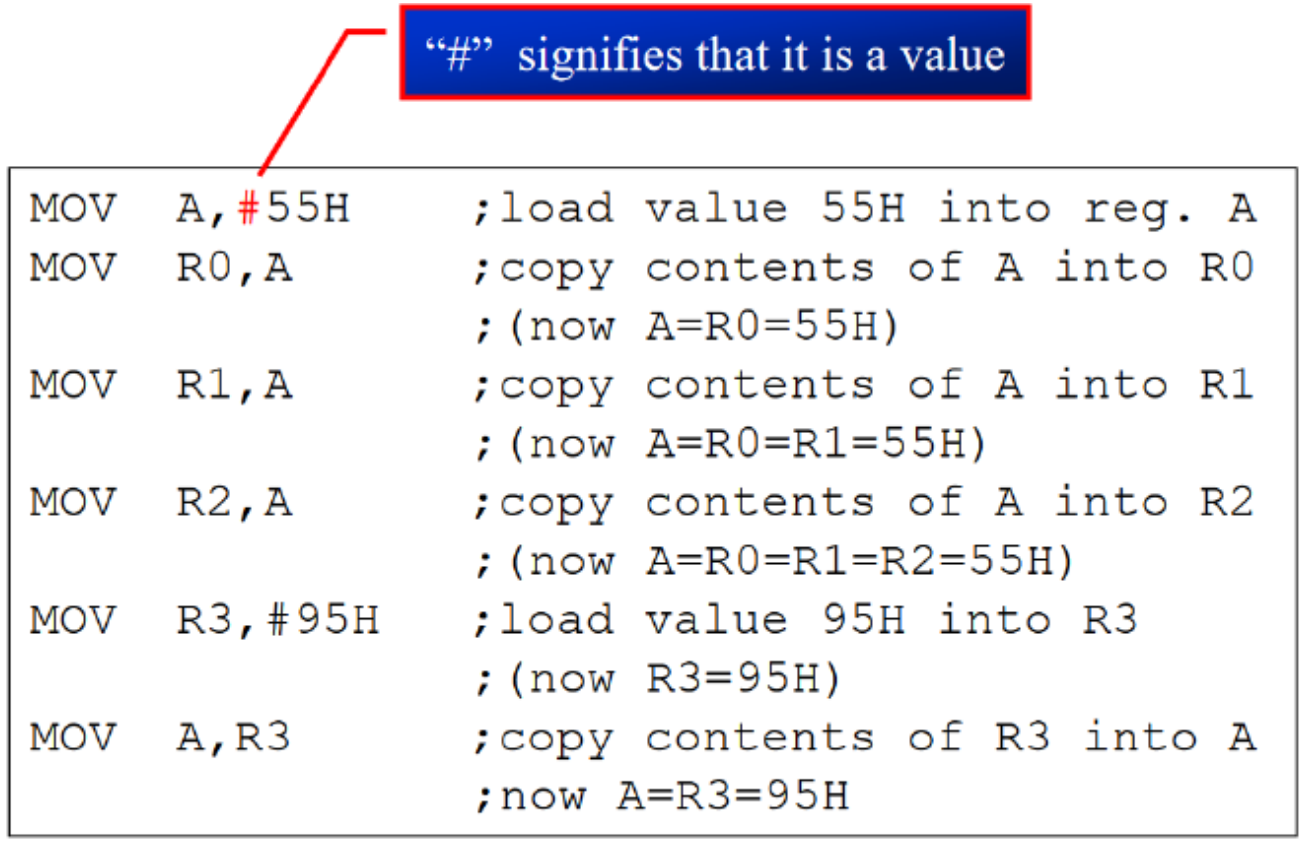
Comments may be at the end of a line or on a line by themselves
The assembler ignores comments

# Assembly Language Programming

## MOV Instruction

**MOV Destination, Source** ;copy source to destination

**The instruction tells the CPU to move (in reality, COPY) the source operand to the destination operand.**



# Assembly Language Programming

- `MOV A, #23H`
- `MOV R5, #0F9H`

If it's not preceded with #, it means to load from a memory location

Add a 0 to indicate that F is a hex number and not a letter

- "`MOV A, #5`", the result will be A=05; i.e., A = 00000101 in binary

- `MOV A, #7F2H` ; ILLEGAL: 7F2H>8 bits (FFH)

# Assembly Language Programming

## ADD Instruction

```
ADD A, source     ;ADD the source operand
                  ;to the  accumulator
```

**Source operand can be either a register or immediate data, but the destination must always be register A.**

```
MOV A, #25H          ;load 25H into A
MOV R2, #34H         ;load 34H into R2
ADD A, R2 ;add R2 to Accumulator
                     ;(A = A + R2)
```

# Assembly Language Programming

```
MOV A, #25H          ;load 25H into A
MOV R2, #34H         ;load 34H into R2
ADD A, R2 ;add R2 to Accumulator
                     ;(A = A + R2)
```

There are always many ways to write the same program depending on the register used.

```
MOV A, #25H          ;load one operand
              ;into A (A=25H)
ADD A, #34H          ;add the second
                     ;operand 34H to A
```

# Assembly Language Programming

**Example:**

Show the status of the CY, AC and P flag after the addition of 38H and 2FH in the following instructions.

```
MOV A, #38H
ADD A, #2FH
```

# Assembly Language Programming

**Example:**

Show the status of the CY, AC and P flag after the addition of 38H and 2FH in the following instructions.

```
MOV A, #38H
ADD A, #2FH ;after the addition A=67H, CY=0
```

**Solution:**

| | |
|---|---|
| 38 | 00111000 |
| +2F | 00101111 |
| 67 | 01100111 |

CY = 0 since there is no carry beyond the D7 bit

AC = 1 since there is a carry from the D3 to the D4 bi

P = 1 since the accumulator has an odd number of 1s (it has five 1s)

The Parity flag is set to '1' if the accumulator contains an odd number of '1's, and it is cleared to '0' if the accumulator has an even number of '1's.

# Assembly Language Programming

**Example:**

Show the status of the CY, AC and P flag after the addition of 9CH and 64H in the following instructions.

```
MOV A, #9CH

ADD A, #64H
```

# Assembly Language Programming

**Example:**

Show the status of the CY, AC and P flag after the addition of 9CH and 64H in the following instructions.

```
MOV A, #9CH
ADD A, #64H  ;after the addition A=00H, CY=1
```

**Solution:**

| | |
|---|---|
| 9C | 10011100 |
| + 64 | 01100100 |
| 100 | 00000000 |

CY = 1 since there is a carry beyond the D7 bit

AC = 1 since there is a carry from the D3 to the D4 bi

P = 0 since the accumulator has an even number of 1s (it has zero 1s)

# Assembly Language Programming

**Example:**

Show the status of the CY, AC and P flag after the addition of 88H and 93H in the following instructions.

```
MOV A, #88H
ADD A, #93H  ;after the addition A=1BH, CY=1
```

# Assembly Language Programming

**Example:**

Show the status of the CY, AC and P flag after the addition of 88H and 93H in the following instructions.

```
MOV A, #88H
ADD A, #93H  ;after the addition A=1BH, CY=1
```

**Solution:**

| | | |
|---|---|---|
| | 88 | 10001000 |
| + | 93 | 10010011 |
| | 11B | 00011011 |

CY = 1 since there is a carry beyond the D7 bit

AC = 0 since there is no carry from the D3 to the D4 bi

P = 0 since the accumulator has an even number of 1s (it has four 1s)

# Assembly Language Programming

**PROGRAM 1 – ADDITION**

```
ORG 0000H
MOV A, #05H
MOV R0, #05H
ADD A, R0
H: SJMP H
END
```

**PROGRAM 2 – SUBTRACTION**

```
ORG 0000H
MOV A, #0AH
MOV R0, #05H
SUBB A, R0
H: SJMP H
END
```

# Assembly Language Programming

**PROGRAM 3 – MULTIPLICATION**

```
ORG 0000H
MOV A, #05H
MOV B, #05H
MUL AB
H: SJMP H
END
```

**PROGRAM 4 – DIVISON**

```
ORG 0000H
MOV A, #20H
MOV B, #05H
DIV AB
H: SJMP H
END
```

# Any Questions?

VIT
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

# Thank you!